



Microcontrollers

Rex Joseph

Department of Electrical and
Electronics Engineering

MSP430 Architecture

Status Register (SR)

The status register (SR/R2), used as a source or destination register, can be used in the **register mode only** addressed with **word instructions**.

The remaining combinations of addressing modes are used to support the **constant generator**

Status Register Bits

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved							V	SCG1	SCG0	OSC OFF	CPU OFF	GIE	N	Z	C
rw-0							rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0	rw-0

MSP430 Architecture

Bit Description

V Overflow bit. This bit is set when the result of an arithmetic operation overflows the signed-variable range.

ADD(.B), ADDC(.B)

Set when:

Positive + Positive = Negative

Negative + Negative = Positive

Otherwise reset

SUB(.B), SUBC(.B), CMP(.B)

Set when:

Positive – Negative = Negative

Negative – Positive = Positive

Otherwise reset

SCG1 System clock generator 1. When set, turns off the SMCLK.

SCG0 System clock generator 0. When set, turns off the DCO dc generator, if DCOCLK is not used for MCLK or SMCLK.

MSP430 Architecture

Bit Description

OSCOFF Oscillator Off. When set, turns off the LFXT1 crystal oscillator, when LFXT1CLK is not in use for MCLK or SMCLK.

CPUOFF CPU off. When set, turns off the CPU.

GIE General interrupt enable. When set, enables maskable interrupts. When reset, all maskable interrupts are disabled. (Also called **Global Interrupt Enable**)

N Negative bit. Set when the result of a byte or word operation is negative and cleared when the result is not negative.

Word operation: N is set to the value of **bit 15** of the result.

Byte operation: N is set to the value of **bit 7** of the result.

Z Zero bit. Set when the result of a byte or word operation is 0 and cleared when the result is not 0.

C Carry bit. Set when the result of a byte or word operation produced a carry and cleared when no carry occurred.

MSP430 Architecture

Constant Generator Registers CG1 and CG2

Six commonly-used constants are generated with the constant generator registers R2 and R3, **without requiring an additional 16-bit word of program code.**

The constants are selected with the **source-register addressing modes (As)**

Register	As	Constant	Remarks
R2	00	-----	Register mode
R2	01	(0)	Absolute address mode
R2	10	00004h	+4, bit processing
R2	11	00008h	+8, bit processing
R3	00	00000h	0, word processing
R3	01	00001h	+1
R3	10	00002h	+2, bit processing
R3	11	0FFFFh	-1, word processing

MSP430 Architecture

The constant generator advantages are:

- No special instructions required
- No additional code word for the six constants
- No code memory access required to retrieve the constant

The assembler uses the constant generator **automatically if one of the six constants is used as an immediate source operand.**

Registers R2 and R3, used in the constant mode, cannot be addressed explicitly; they act as source-only registers.

3.4.1 Double-Operand (Format I) Instructions

Figure 3-9 illustrates the double-operand instruction format.

Figure 3-9. Double Operand Instruction Format

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Op-code				S-Reg				Ad	B/W	As		D-Reg			

MSP430 Architecture

Constant Generator - Expanded Instruction Set

The RISC instruction set of the MSP430 has only **27 instructions**.

However, the constant generator allows the MSP430 assembler to support 24 additional, **emulated instructions**.

For example, the single-operand instruction

CLR dst

is emulated by the double-operand instruction with the same length:

MOV R3,dst

where the #0 is replaced by the assembler, and R3 is used with As=00.

INC dst

is replaced by:

ADD 0(R3),dst

Details later!

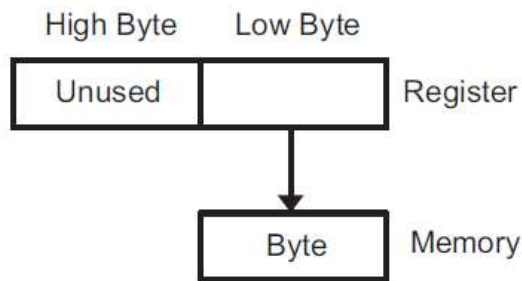
MSP430 Architecture

General-Purpose Registers R4 to R15

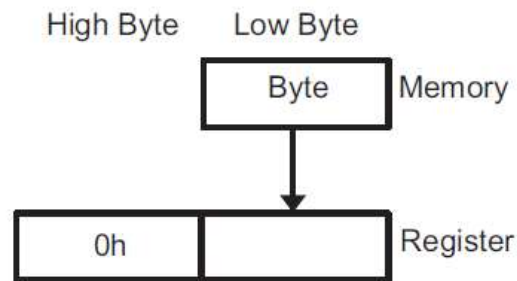
The twelve registers, R4-R15, are general-purpose registers.

All of these registers can be used as data registers, address pointers, or index values and can be accessed with **byte** or **word** instructions

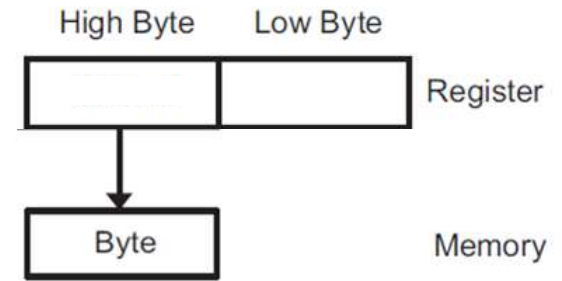
Register-Byte Operation



Byte-Register Operation



Register-Byte Operation



MSP430 Architecture

Oscillator and System Clock

The clock system is supported by the basic clock module that includes support for a 32768-Hz watch crystal oscillator, an internal very-low-power low-frequency oscillator and an internal **digitally controlled oscillator (DCO)**.

The basic clock module is designed to meet the requirements of both low system cost and low power consumption.

The internal DCO provides a fast turn-on clock source and stabilizes in less than 1 μ s.

MSP430 Architecture

The basic clock module provides the following clock signals:

- Auxiliary clock (ACLK), sourced either from a 32768-Hz watch crystal or the internal LF oscillator.
- Main clock (MCLK), the system clock used by the CPU.
- Sub-Main clock (SMCLK), the sub-system clock used by the peripheral modules.

The DCO settings to calibrate the DCO output frequency are stored in the **information memory segment A**.

MSP430 Architecture

Brownout

The brownout circuit is implemented to provide the proper internal reset signal to the device during power on and power off.

Watchdog Timer (WDT+)

The primary function of the watchdog timer (WDT+) module is to perform a controlled system restart after a software problem occurs.

If the selected time interval expires, a system reset is generated.

If the watchdog function is not needed in an application, the module can be **disabled** or **configured as an interval timer** and can generate **interrupts at selected time intervals**.

MSP430 Architecture

Digital I/O

Up to three 8-bit I/O ports are implemented:

- All individual I/O bits are independently programmable.
- Any combination of input, output, and interrupt condition (port P1 and port P2 only) is possible.
- Edge-selectable interrupt input capability for all bits of port P1 and port P2 (if available).
- Read/write access to port-control registers is supported by all instructions.
- Each I/O has an individually programmable pull-up or pull-down resistor.
- Each I/O has an individually programmable **pin oscillator enable bit** to enable low-cost **capacitive touch detection**.

Operating Modes

Operating Modes

The MSP430 has **one active** mode and **five software selectable low-power modes** of operation.

An interrupt event can wake up the device from any of the low-power modes, service the request, and restore back to the low-power mode on return from the interrupt program.

The following six operating modes can be configured by software:

- Active mode (AM)
 - All clocks are active
- Low-power mode 0 (LPM0)
 - CPU is disabled
 - ACLK and SMCLK remain active, MCLK is disabled

Operating Modes

- Low-power mode 1 (LPM1)
 - CPU is disabled
 - ACLK and SMCLK remain active, MCLK is disabled
 - DCO's dc generator is disabled if DCO not used in active mode
- Low-power mode 2 (LPM2)
 - CPU is disabled
 - MCLK and SMCLK are disabled
 - DCO's dc generator remains enabled
 - ACLK remains active

Operating Modes

- Low-power mode 3 (LPM3)
 - CPU is disabled
 - MCLK and SMCLK are disabled
 - DCO's dc generator is disabled
 - ACLK remains active

- Low-power mode 4 (LPM4)
 - CPU is disabled
 - ACLK is disabled
 - MCLK and SMCLK are disabled
 - DCO's dc generator is disabled
 - Crystal oscillator is stopped

Operating Modes

- Low-power mode 3 (LPM3)
 - CPU is disabled
 - MCLK and SMCLK are disabled
 - DCO's dc generator is disabled
 - ACLK remains active
- Low-power mode 4 (LPM4)
 - CPU is disabled
 - ACLK is disabled
 - MCLK and SMCLK are disabled
 - DCO's dc generator is disabled
 - Crystal oscillator is stopped

Addressing Modes

The instruction set consists of the original 51 instructions with **three formats** and **seven address modes** and additional instructions for the expanded address range.

Each instruction can operate on word and byte data.

INSTRUCTION FORMAT	EXAMPLE	OPERATION
Dual operands, source-destination	ADD R4,R5	$R4 + R5 \rightarrow R5$
Single operands, destination only	CALL R8	$PC \rightarrow (TOS), R8 \rightarrow PC$
Relative jump, un/conditional	JNE	Jump-on-equal bit = 0

ADDRESS MODE	S	D	SYNTAX	EXAMPLE	OPERATION
Register	✓	✓	MOV Rs,Rd	MOV R10,R11	$R10 \rightarrow R11$
Indexed	✓	✓	MOV X(Rn),Y(Rm)	MOV 2(R5),6(R6)	$M(2+R5) \rightarrow M(6+R6)$
Symbolic (PC relative)	✓	✓	MOV EDE,TONI		$M(EDE) \rightarrow M(TONI)$
Absolute	✓	✓	MOV &MEM,&TCDAT		$M(MEM) \rightarrow M(TCDAT)$
Indirect	✓		MOV @Rn,Y(Rm)	MOV @R10,Tab(R6)	$M(R10) \rightarrow M(Tab+R6)$
Indirect autoincrement	✓		MOV @Rn+,Rm	MOV @R10+,R11	$M(R10) \rightarrow R11$ $R10 + 2 \rightarrow R10$
Immediate	✓		MOV #X,TONI	MOV #45,TONI	$\#45 \rightarrow M(TONI)$

Addressing modes

Addressing Modes

Seven addressing modes for the source operand and four addressing modes for the destination operand can address the complete address space with no exceptions. **Rem: Address and contents of the addressed location**

The bit numbers in Table describe the contents of the As (source) and Ad (destination) mode bits.

As/Ad	Addressing Mode	Syntax	Description
00/0	Register mode	Rn	Register contents are operand
01/1	Indexed mode	X(Rn)	(Rn + X) points to the operand. X is stored in the next word.
01/1	Symbolic mode	ADDR	(PC + X) points to the operand. X is stored in the next word. Indexed mode X(PC) is used.
01/1	Absolute mode	&ADDR	The word following the instruction contains the absolute address. X is stored in the next word. Indexed mode X(SR) is used.
10/-	Indirect register mode	@Rn	Rn is used as a pointer to the operand.
11/-	Indirect autoincrement	@Rn+	Rn is used as a pointer to the operand. Rn is incremented afterwards by 1 for .B instructions and by 2 for .W instructions.
11/-	Immediate mode	#N	The word following the instruction contains the immediate constant N. Indirect autoincrement mode @PC+ is used.

Addressing modes

Register Mode

Assembler Code	Content of ROM
MOV R10,R11	MOV R10,R11

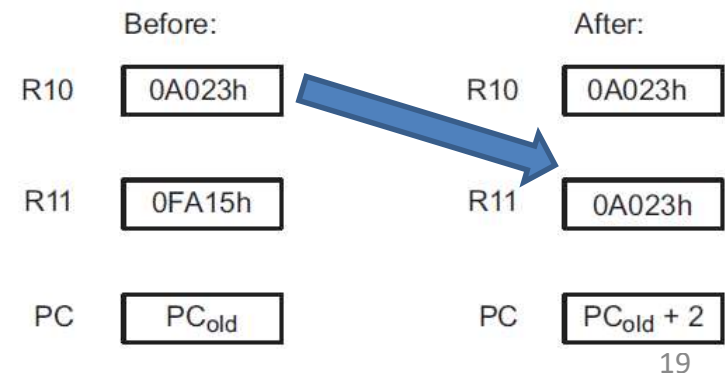
Length: One or two words
 Operation: Move the content of R10 to R11. R10 is not affected.
 Comment: Valid for source and destination

Data in Registers

The data in the register can be accessed using word or byte instructions.

If byte instructions are used, the high byte is always 0 in the result.

The status bits are handled according to the result of the byte instructions.



Addressing modes

Indexed Mode

Assembler Code	Content of ROM
MOV 2 (R5) , 6 (R6)	MOV X (R5) , Y (R6)
	X = 2
	Y = 6

Length: Two or three words

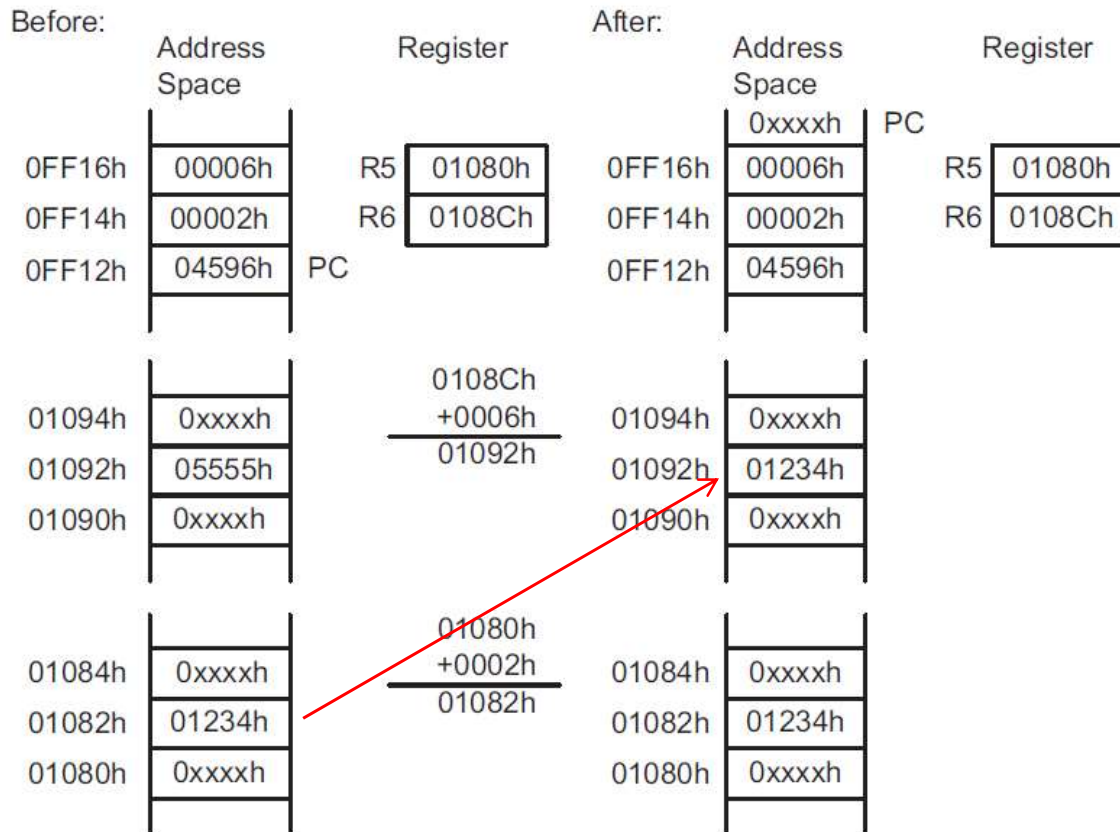
Operation: Move the contents of the **source address** (contents of R5 + 2) to the **destination address** (contents of R6 + 6). The source and destination registers (R5 and R6) are not affected.

Comment: Valid for source and destination

In indexed mode, the program counter is incremented automatically so that program execution continues with the next instruction.

Addressing modes

Indexed Mode MOV 2(R5),6(R6);



Addressing modes

Symbolic Mode

Assembler Code	Content of ROM
MOV EDE, TONI	MOV X(PC), Y(PC) X = EDE - PC Y = TONI - PC

Length: Two or three words

Operation: Move the contents of the source **address EDE (contents of PC + X)** to the destination address TONI (contents of PC + Y). The words after the instruction contain the differences between the PC and the source or destination addresses. The assembler computes and inserts offsets X and Y automatically. With symbolic mode, the program counter (PC) is incremented automatically so that program execution continues with the next instruction.

Comment: Valid for source and destination

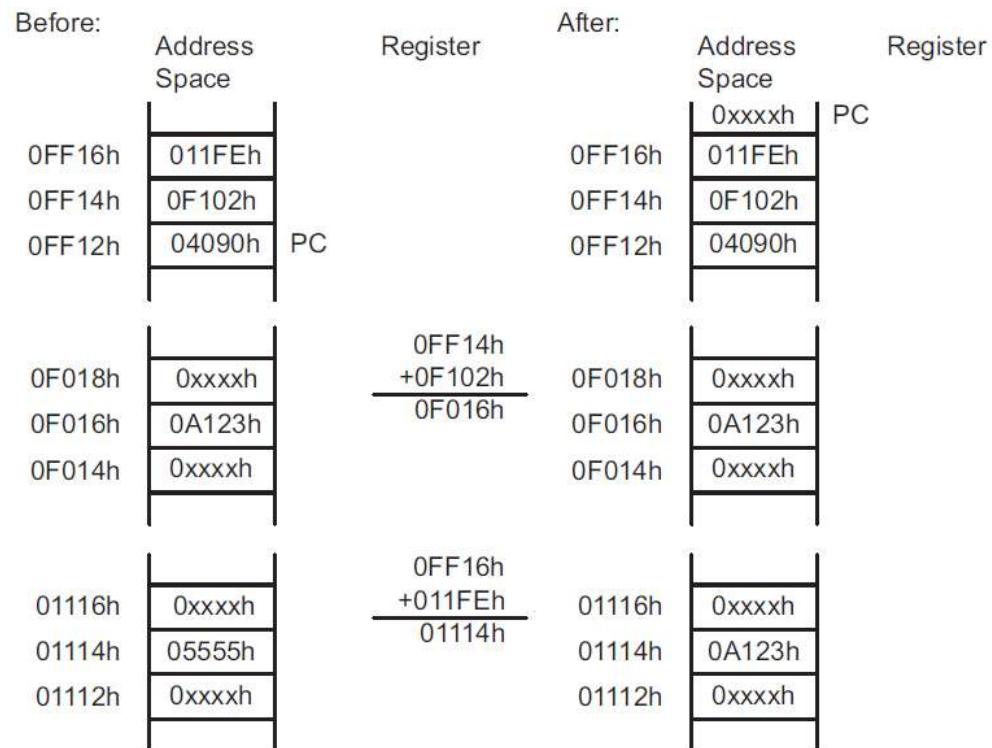
Addressing modes

Symbolic Mode

MOV EDE,TONI ;Source address EDE = 0F016h
;Dest. address TONI = 01114h

Move the contents of the source address EDE (contents of PC + X) to the destination address TONI (contents of PC+ Y).

The words after the instruction contain the differences between the PC and the source or destination addresses.



Addressing modes

Symbolic Mode

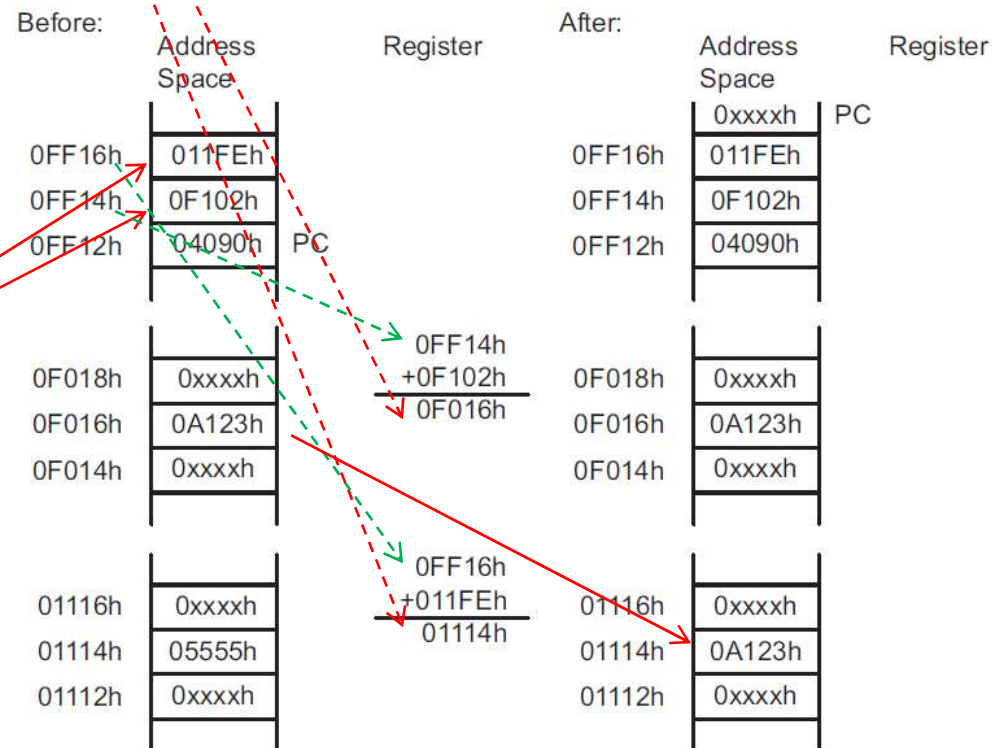
MOV EDE,TONI

;Source address EDE = 0F016h

;Dest. address TONI = 01114h

Move the contents of the source address EDE (contents of PC + X) to the destination address TONI (contents of PC + Y).

The words after the instruction contain the differences between the PC and the source or destination addresses.



Addressing modes

Absolute Mode

Assembler Code	Content of ROM
MOV &EDE, &TONI	MOV X(0), Y(0) X = EDE Y = TONI

Length: Two or three words

Operation: Move the contents of the source address EDE to the destination address TONI. The words after the instruction contain the absolute address of the source and destination addresses. With absolute mode, the PC is incremented automatically so that program execution continues with the next instruction.

Comment: Valid for source and destination

This address mode is mainly for hardware peripheral modules that are located at an absolute, fixed address. These are addressed with absolute mode to ensure software transportability (for example, position-independent code).

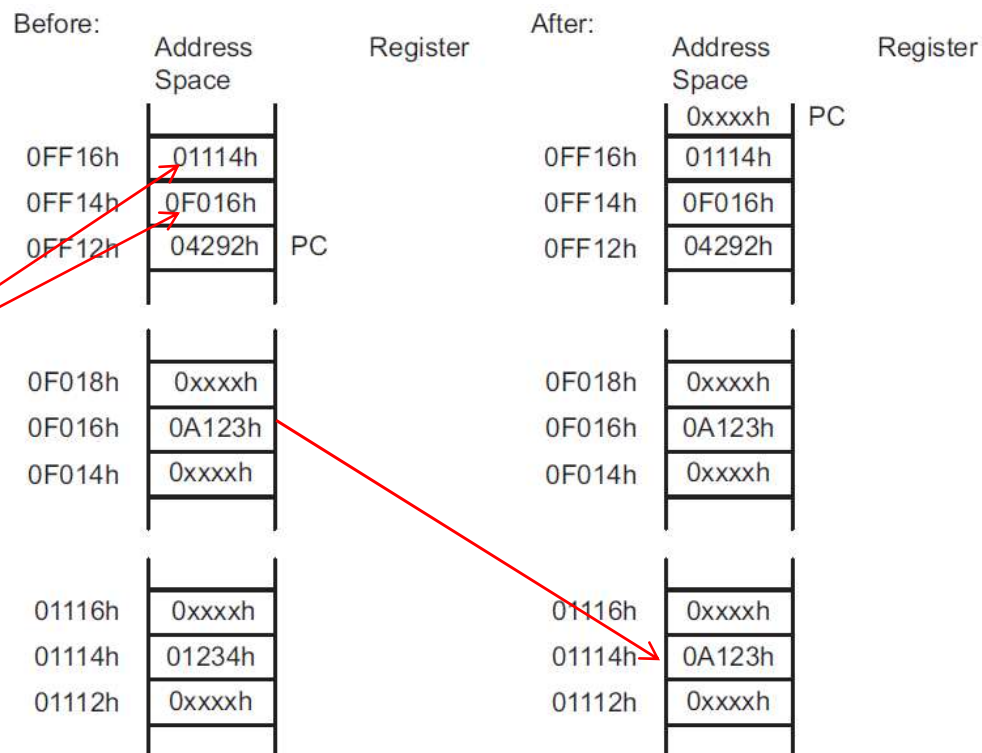
Addressing modes

Absolute Mode

MOV &EDE,&TONI ;Source address EDE = 0F016h
;Dest. address TONI = 01114h

Move the contents of the source address EDE to destination address TONI.

The words after the instruction contain the absolute address of the source and destination addresses.



Addressing modes

Indirect Register Mode

Assembler Code	Content of ROM
MOV @R10, 0(R11)	MOV @R10, 0(R11)

Length: One or two words

Operation: Move the **contents of the source address** (contents of R10) to the **destination address** (contents of R11). The registers are not modified.

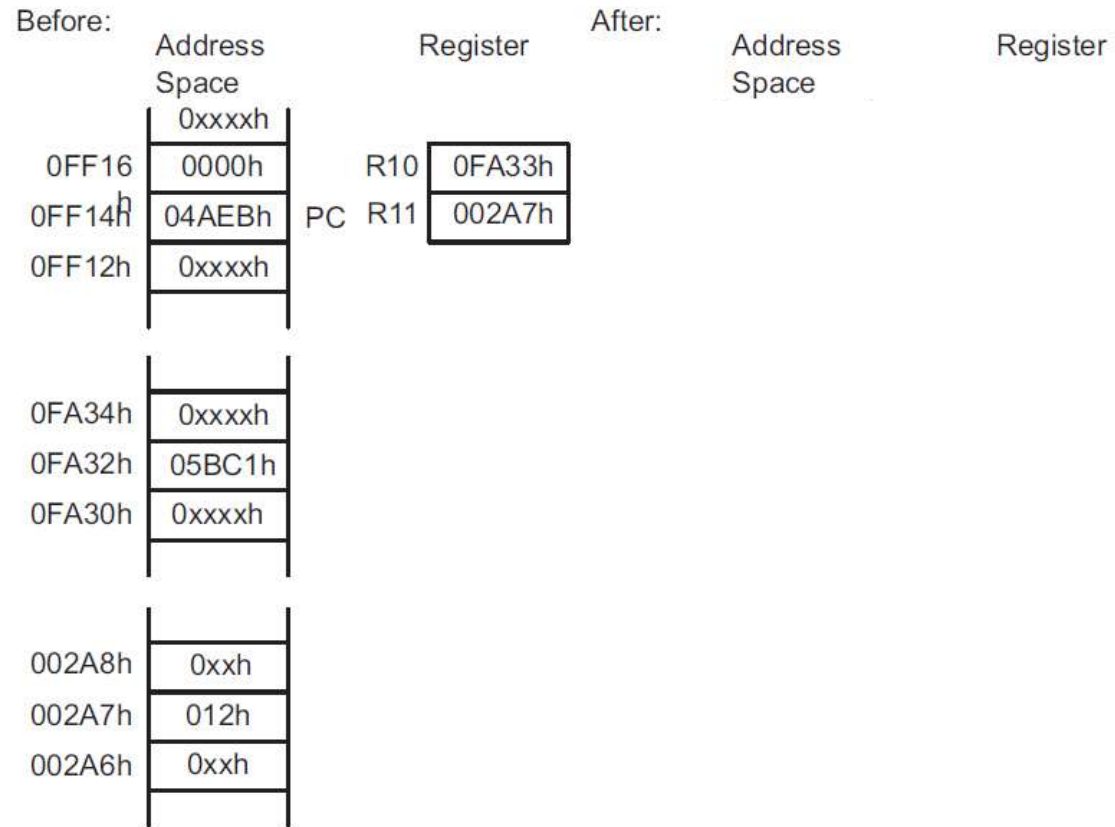
Comment: Valid **only for source operand**. The substitute for destination operand is **0(Rd)**.

Addressing modes

Indirect Register Mode

MOV.B @R10,0(R11)

Move the contents of the source address (contents of R10) to the destination address (contents of R11). The registers are not modified.

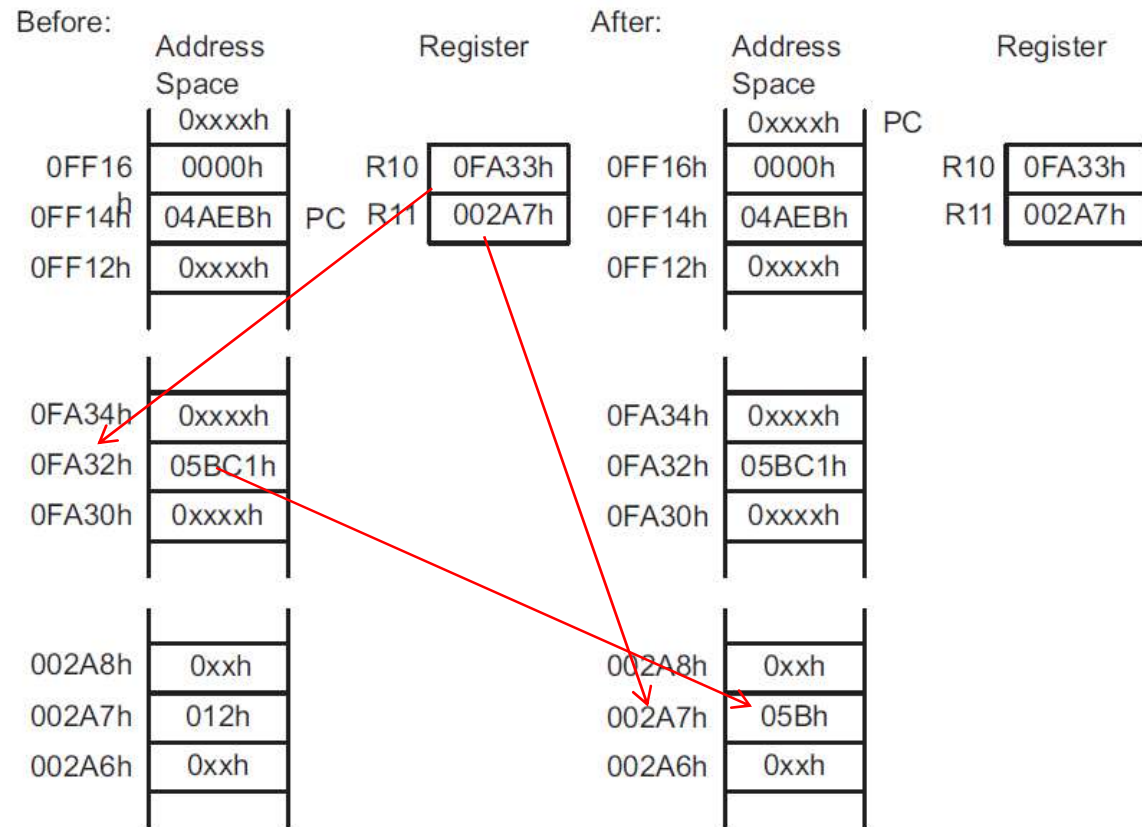


Addressing modes

Indirect Register Mode

MOV.B @R10,0(R11)

Move the contents of the source address (contents of R10) to the destination address (contents of R11). The registers are not modified.



Addressing modes

Indirect Auto-increment Mode

Assembler Code	Content of ROM
MOV @R10+, 0(R11)	MOV @R10+, 0(R11)

Length: One or two words

Operation: Move the contents of the source address (contents of R10) to the destination address (contents of R11). Register **R10** is incremented by **1 for a byte** operation, or **2 for a word** operation after the fetch; it points to the **next address** without any overhead. This is **useful for table processing**.

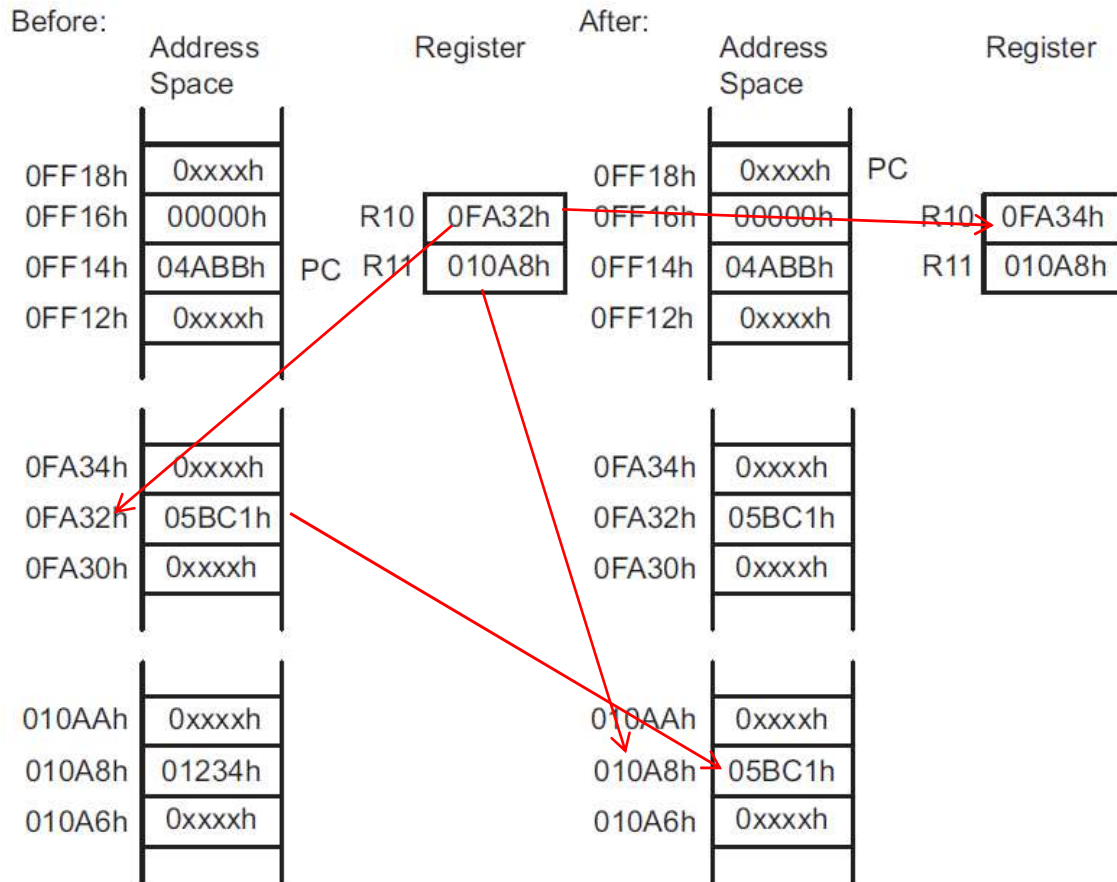
Comment: Valid only for source operand. The substitute for destination operand is 0(Rd) plus second instruction INCD Rd

Addressing modes

Indirect Auto-increment Mode

MOV @R10+,0(R11)

Move the contents of the source address (contents of R10) to the destination address (contents of R11). Register R10 is incremented by 1 for a byte operation, or 2 for a word operation after the fetch; it points to the next address



Addressing modes

Immediate Mode

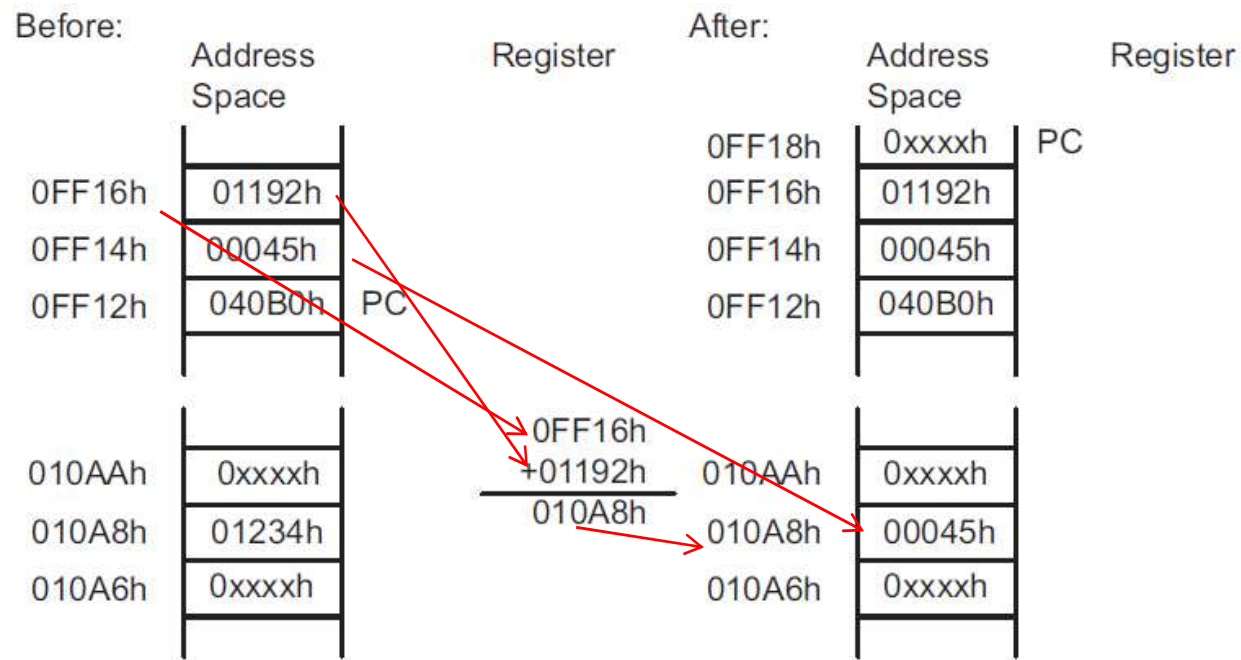
Assembler Code	Content of ROM
MOV #45h, TONI	MOV @PC+, X(PC)
	45
	$X = \text{TONI} - \text{PC}$

- Length: Two or three words
It is one word less if a constant of CG1 or CG2 can be used.
- Operation: Move the immediate constant 45h, which is contained in the **word following the instruction**, to destination address TONI. When fetching the source, the program counter points to the word following the instruction and moves the contents to the destination.
- Comment: Valid only for a source operand

Addressing modes

Immediate Mode

MOV #45h,TONI



Instruction Set

Instruction Set

The complete MSP430 instruction set consists of 27 core instructions and 24 emulated instructions.

The **core instructions** are instructions that have **unique op-codes** decoded by the CPU.

The **emulated instructions** are instructions that make code easier to write and read, but **do not have op-codes themselves**, instead they are **replaced** automatically by the **assembler** with an **equivalent core instruction**.

There is no code or performance penalty for using emulated instruction.

Instruction Set

There are three core instruction formats:

- Dual operand
- Single operand
- Jump

Instruction Set

All single-operand and dual-operand instructions can be byte or word instructions by using .B or .W extensions.

Byte instructions are used to access byte data or byte peripherals.

Word instructions are used to access word data or word peripherals.
If **no extension** is used, the instruction is a **word instruction**.

Instruction Set

The source and destination of an instruction are defined by the following fields:

src	The source operand defined by As and S-reg
dst	The destination operand defined by Ad and D-reg
As	The addressing bits responsible for the addressing mode used for the source (src)
S-reg	The working register used for the source (src)
Ad	The addressing bits responsible for the addressing mode used for the destination (dst)
D-reg	The working register used for the destination (dst)

B/W Byte or word operation:

0: word operation

1: byte operation

Instruction Set

Destination Address

Destination addresses are valid anywhere in the memory map.

However, when using an instruction that modifies the contents of the destination, the user must ensure the destination address is writable.

For example, a **masked-ROM** location would be a **valid destination address**, but the contents are not modifiable, so the results of the instruction would be lost.



THANK YOU

Rex Joseph

Department of Electrical and Electronics Engineering

rexjoseph@pes.edu

+91 80 6666 3333 Extn 515